

Enterprise System Integration

Assignment #3 System Design

University Student Information System

Team Members

Hashim
Sudais
Calvin
Daboikiabo

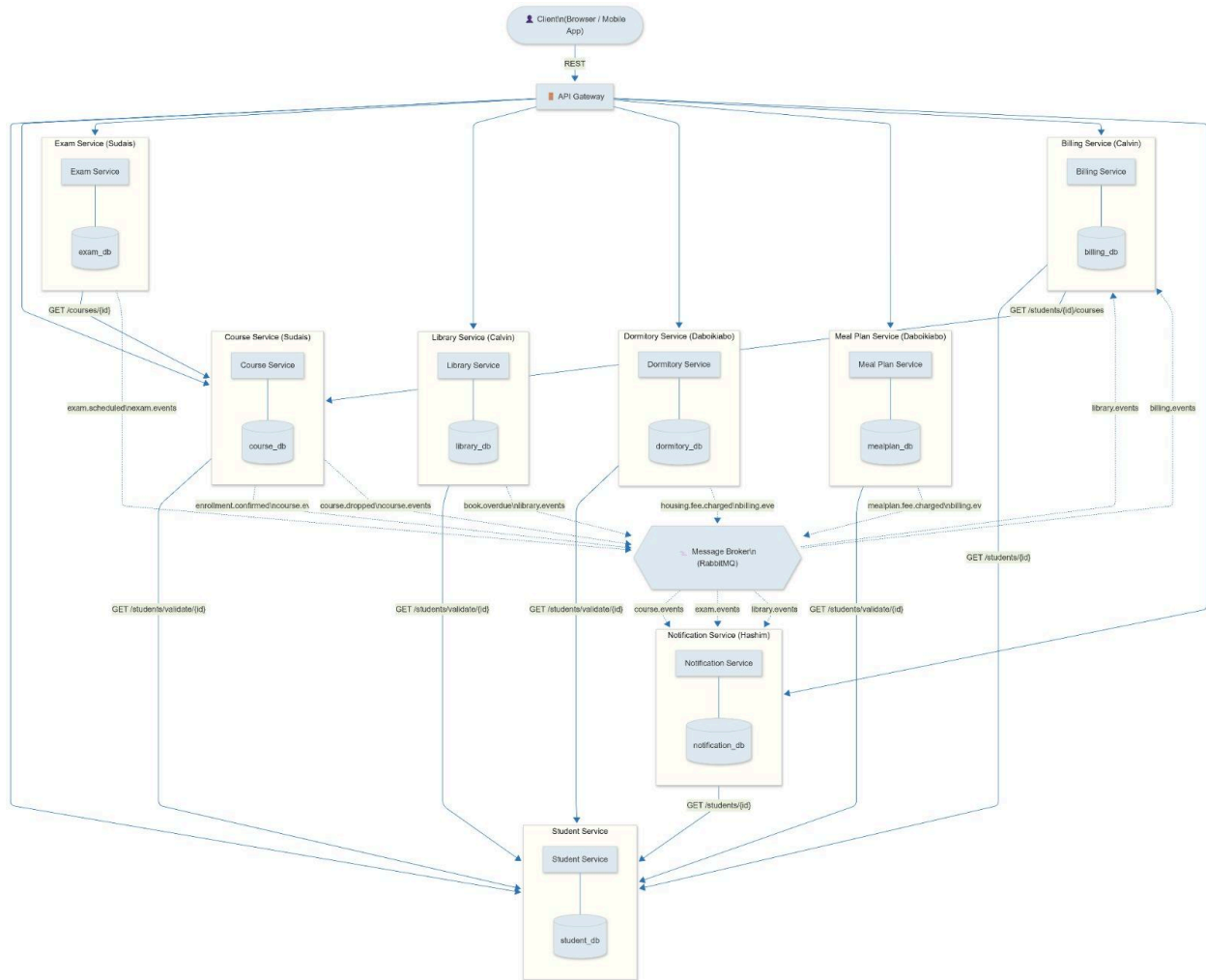
1. Complete System Architecture

The University Student Information System is composed of eight microservices, each owning its own domain and database. Services communicate via REST (synchronous) and a message broker (asynchronous) to remain loosely coupled.

1.1 Service Overview

Service	Responsibilities	Key Relationships
Student Service	Manage student registration, profile, and identity verification	Used by all other services for auth
Course Service	Create courses, manage enrollment, track credits	Depends on Student; used by Exam, Billing
Exam Service	Schedule exams linked to courses and students	Depends on Course, Student; uses Notification
Library Service	Book catalog, borrowing, fines, overdue tracking	Depends on Student; uses Notification, Billing
Dormitory Service	Room management, room assignment per semester	Depends on Student; uses Billing
Meal Plan Service	Meal plan catalog and student subscriptions	Depends on Student; uses Billing
Billing Service	Aggregate charges: tuition, housing, meals, fines	Depends on Course; receives charges from others
Notification Service	Deliver email / in-app messages to students	Depends on Student; consumed by all services

1.2 Architecture Diagram



1.3 Scope of Implementation

All eight services are part of the full system design. The table below confirms which components will be implemented during the project phase. No service is replaced by a shared infrastructure component — the message broker (RabbitMQ) is a shared platform-level concern used by all services.

Service	Owner	Status	Role in System
Student Service	Hashim	✓ Will be implemented	Central identity provider called synchronously by all other services
Notification Service	Hashim	✓ Will be implemented	Async event consumer; delivers alerts to students
Course Service	Sudais	✓ Will be implemented	Enrollment orchestrator; produces enrollment events
Exam Service	Sudais	✓ Will be implemented	Exam scheduler; produces exam.scheduled event
Library Service	Calvin	✓ Will be implemented	Borrowing lifecycle; produces book.overdue event
Billing Service	Calvin	✓ Will be implemented	Financial aggregator; consumes charge events
Dormitory Service	Daboikiabo	✓ Will be implemented	Room assignments; produces housing.fee.charged event
Meal Plan Service	Daboikiabo	✓ Will be implemented	Plan subscriptions; produces mealplan.fee.charged event

2. Service APIs

2.1 Student Service (Hashim)

Each endpoint is consistent with R1–R3 and the domain model (Student aggregate root). The Student Service acts as the central identity provider for all other services in the system.

Endpoint	Method	Description
/students	POST	Register a new student with personal and academic information (admin only)
/students/{studentId}	GET	Retrieve a student's full profile by student ID
/students/{studentId}	PUT	Update a student's contact information and profile details
/students/{studentId}	DELETE	Deactivate or remove a student account (admin only)

Endpoint	Method	Description
/students/{studentId}/status	PATCH	Update a student's academic status — active, suspended, graduated, on leave (admin only)
/students/validate/{studentId}	GET	Validate that a student exists and is currently active (called internally by other services for identity verification)

2.2 Notification Service (Hashim)

Each endpoint is consistent with R30–R31 and the domain model (INotificationService domain service). The Notification Service is a pure cross-cutting concern — it receives notification requests from other services and delivers alerts via email or in-app messaging.

Endpoint	Method	Description
/notifications	POST	Request delivery of a notification to a student (called by Course, Exam, Library, and other services)
/notifications/{notificationId}	GET	Retrieve a specific notification record by ID
/notifications/student/{studentId}	GET	Retrieve all notifications sent to a specific student
/notifications/{notificationId}/status	GET	Check the current delivery status of a notification (pending, sent, failed)
/notifications/templates	GET	List all available message templates
/notifications/templates	POST	Create a new message template (admin only)
/notifications/templates/{templateId}	PUT	Update an existing message template

=====

2.3 Course Service (Sudais): Each endpoint is consistent with R4–R8 and the domain model (Course aggregate + enrollments).

Endpoint	Method	Description
/courses	POST	Create a new course (admin only – requires course details, capacity, instructor)
/courses	GET	Retrieve list of all available courses (with optional filters: status, semester)
/courses/{courseId}	GET	Retrieve detailed information about a specific course

/courses/{courseId}/enroll	POST	Enroll a student in a course (verifies identity/eligibility synchronously with Student Service)
/courses/{courseId}/enroll/{studentId}	DELETE	Drop a student from a course
/students/{studentId}/courses	GET	Get all courses a student is enrolled in (used by students and other services)

2.4 Exam Service (Sudais): Each endpoint is consistent with R9–R12 and the domain model (ExamSchedule aggregate linked to Course).

Endpoint	Method	Description
/exams	POST	Create an exam schedule linked to an existing course (retrieves course data synchronously from Course Service)
/exams	GET	Retrieve list of all exam schedules (with optional filters: courseId, date range)
/exams/{examId}	GET	Retrieve details of a specific exam schedule
/students/{studentId}/exams	GET	Get exam schedules for a student's enrolled courses
/exams/{examId}/notify	POST	Trigger notification to all enrolled students for an exam (asynchronous via Notification Service)
/courses/{courseId}/exams	GET	Get all exam schedules linked to a specific course

2.5 Library Service (Calvin)

Base URL: /api/library

Endpoint	Method	Description
/books	POST	R13 — Add a new book (library staff)
/books/{id}	PUT	R13 — Update book availability
/books	GET	Browse the book catalog
/books/{id}/borrow	POST	R14 — Borrow a book (verifies student identity)
/loans/{id}/return	PUT	R15 — Return a borrowed book
/loans/student/{studentId}	GET	View active loans for a student

2.6 Billing Service (Calvin)

Base URL: /api/billing

Endpoint	Method	Description
----------	--------	-------------

/billing/{studentId}/charges	GET	R26 — View aggregated charges (finance office)
/billing/{studentId}/tuition	POST	R27 — Calculate tuition from enrolled credits (calls Course Service)
/billing/{studentId}/status	GET	R28 — View payment status and outstanding balance
/billing/{studentId}/pay	POST	R29 — Simulate payment (dummy endpoint)
/billing/{studentId}/charges	POST	Add a charge (housing, meal, fine) — called by other services

2.7 Dormitory Service (Daboikiabo)

Base URL: /api/dormitory

Each endpoint is consistent with the domain model (Room aggregate root, RoomAssignment aggregate). The Dormitory Service manages room inventory and semester-based student assignments.

Endpoint	Method	Description
/rooms	POST	Add a new room to the inventory (admin only requires room number, building, capacity, amenities)
/rooms	GET	Retrieve list of all rooms with optional filters (building, capacity, is_available)
/rooms/{roomId}	GET	Retrieve detailed information about a specific room
/rooms/{roomId}/assign	POST	Assign a student to a room for the current semester (verifies student identity synchronously with Student Service)
/assignments/student/{studentId}	GET	Get the current room assignment for a specific student

/assignments/{assignmentId}	DELETE	Remove a student's room assignment (e.g., move-out, withdrawal; triggers billing adjustment via async event)
/rooms/{roomId}/availability	GET	Check room availability and current occupancy
/students/{studentId}/eligibility	GET	Validate if a student is eligible for housing (active status, no existing assignment, called internally by other services)

2.8 Meal Plan Service (Daboikiabo)

Base URL: /api/meal-plan

Each endpoint is consistent with the domain model (MealPlan aggregate root, Subscription entity). The Meal Plan Service manages plan catalog and student subscriptions per semester.

Endpoint	Method	Description
/plans	POST	Create a new meal plan type (admin only, requires a name, meal count per week, price)
/plans	GET	Retrieve list of all available meal plans
/plans/{planId}	GET	Retrieve detailed information about a specific meal plan
/plans/{planId}/subscribe	POST	Subscribe a student to a meal plan for the current semester (verifies student identity synchronously with Student Service)
/subscriptions/student/{studentId}	GET	Get the current meal plan subscription for a specific student
/subscriptions/{subscriptionId}	DELETE	Cancel a student's meal plan subscription (triggers billing adjustment via async event)
/students/{studentId}/eligibility	GET	Validate if a student is eligible

3.2 Notification Service Data Model

This microservice manages its own database. It stores notification delivery records and reusable message templates.

Entity	Attributes	Type	Relationships
Notification	notificationId (UUID, PK)	Aggregate Root	References studentId (external, no FK constraint)
Notification	recipientId (UUID)	Attribute	studentId from Student Service
Notification	channel (Enum: email, in_app)	Attribute	
Notification	subject, body (String)	Attribute	
Notification	status (Enum: pending, sent, failed)	Attribute	
Notification	sentAt, createdAt (DateTime)	Attribute	
NotificationTemplate	templateId (UUID, PK)	Entity (audit record)	Used by Notification to format messages
NotificationTemplate	name, subject, bodyTemplate (String)	Attribute	

3.3 Course Service Data Model: This microservice manages its own database. No direct sharing with other services.

Entities & Attributes (ER-style):

- **Course** (Aggregate Root)
 - courseId (PK, UUID)
 - name (String)
 - instructor (String)
 - capacity (Integer)
 - status (Enum: OPEN, CLOSED, CANCELLED)
 - enrolledCount (Integer, derived)
 - semester (String)
- **Enrollment** (Entity inside Course aggregate)
 - enrollmentId (PK, UUID)
 - courseId (FK)
 - studentId (String – reference only, no full student data)
 - enrollmentDate (Date)
 - status (Enum: ACTIVE, DROPPED)

Relationships:

- One Course can have many Enrollments (1-to-many).
- Enrollment references studentId only (identity verification is done via synchronous call to Student Service).

3.4 Exam Service Data Model: This microservice manages its own database.

Entities & Attributes (ER-style):

- **ExamSchedule** (Aggregate Root)
 - examId (PK, UUID)
 - courseId (FK – reference to Course Service)
 - date (Date)
 - time (String)
 - location (String)
 - duration (Integer – minutes)
 - status (Enum: SCHEDULED, CANCELLED)

Relationships:

- One ExamSchedule is linked to exactly one Course (many-to-one).
- No direct student data is stored here; enrolled students are retrieved on demand from Course Service when notifications are triggered.

3.5 Library Service - library_db

Entity	Attribute	Type	Relationship
books	book_id (PK, UUID)	UUID	1-to-many with BookLoan
	title	VARCHAR(255)	
	is_available	BOOLEAN	
book_loans	loan_id (PK, UUID)	UUID	Many-to-one with Book; references studentId (external)
	book_id (FK)	UUID	
	student_id	UUID	
	due_date	DATE	
	returned_at	TIMESTAMP	

3.6 Billing Service - billing_db

Entity	Attribute	Type	Relationship
billing_accounts	account_id (PK, UUID)	UUID	References studentId (external); 1-to-many with Charge and Payment
	student_id	UUID	
	outstanding_balance	DECIMAL(10,2)	
charges	charge_id (PK, UUID)	UUID	Many-to-one with BillingAccount
	account_id (FK)	UUID	
	charge_type	VARCHAR(30)	
	amount	DECIMAL(10,2)	
	created_at	TIMESTAMP	
Payment	paymentId (UUID, PK)	Entity	Many-to-one with BillingAccount
	amount (Decimal), method (String)	Attribute	
	paymentDate (DateTime)	Attribute	

3.7 Data Model (Dormitory Service)

This microservice manages its own database (`dormitory_db`). No direct sharing with other services. Other services reference `studentId` only as a logical foreign key.

Entity	Attributes	Type	Relationships
Room (Aggregate Root)	roomId (PK)	UUID	One-to-many with RoomAssignment
	roomNumber	String	
	building	String	
	capacity	Integer	
	currentOccupancy	Integer (derived)	

	amenities	List(String)	
	isAvailable	Boolean	
RoomAssignment (Entity)	assignmentId (PK)	UUID	Many-to-one with Room; references studentId (external)
	roomId (FK)	UUID	
	studentId	UUID	
	semester	String	
	assignmentDate	Date	
	status	Enum: ACTIVE, COMPLETED, CANCELLED	

3.8 Data Model (Meal Service)

This microservice manages its own database (mealplan_db). No direct sharing with other service.

Entity	Attributes	Type	Relationships
MealPlan (Aggregate Root)	planId (PK)	UUID	One-to-many with Subscription
	name	string	
	mealsPerWeek	integer	
	price	Decimal (10, 2)	
	semester	string	
	isActive	boolean	
Subscription	subscriptionId (PK)	uuid	
	planId (FK)	UUID	
	studentId	UUID (reference only)	
	startDate	Date	
	endDate	Date	

	status	Enum: ACTIVE, COMPLETED, CANCELLED	
--	--------	--	--

4. System Workflows

Workflow 1: Student Course Enrollment

This workflow demonstrates synchronous REST communication combined with asynchronous event-driven notification. It is triggered when a student requests enrollment in a course.

Services Involved

- Course Service (orchestrator)
- Student Service (identity verification)
- Notification Service (async alert delivery)

Interaction Sequence

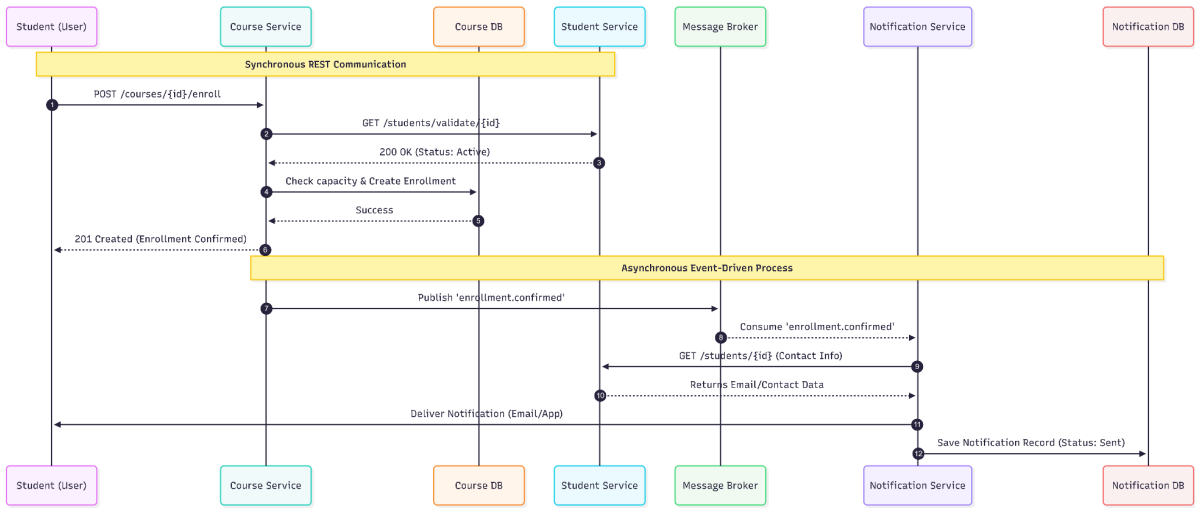
Step	From	To	Action / Event
1	Student (User)	Course Service	POST /courses/{id}/enroll — student submits enrollment request
2	Course Service	Student Service	GET /students/validate/{id} — synchronous REST call to verify student is active and eligible
3	Student Service	Course Service	Returns student status (200 OK or error)
4	Course Service	Course DB	Checks capacity and creates Enrollment record (status: active)
5	Course Service	Message Broker	Publishes enrollment.confirmed event to topic: course.events
6	Notification Service	Message Broker	Consumes enrollment.confirmed event from topic: course.events
7	Notification Service	Student Service	GET /students/{id} — retrieves student contact info for delivery
8	Notification Service	Student (Email/App)	Delivers enrollment confirmation notification; saves Notification record

Data Tables Updated

- Course Service DB: Enrollment table (new record inserted, course enrolled count incremented)
- Notification Service DB: Notification table (record created with status: sent)

Event Details

Event	Topic / Channel	Payload
enrollment.confirmed	course.events	{ studentId, courseId, courseName, enrolledAt }



Workflow 2: Dormitory Room Assignment & Billing

This workflow demonstrates how the Dormitory Service integrates with the Student Service (synchronous identity verification) and the Billing Service (asynchronous housing charge creation). It is triggered when a student applies for a dormitory room.

Services Involved:

- Dormitory Service — orchestrator
- Student Service — synchronous identity verification
- Billing Service — asynchronous housing charge creation

Interaction Sequence:

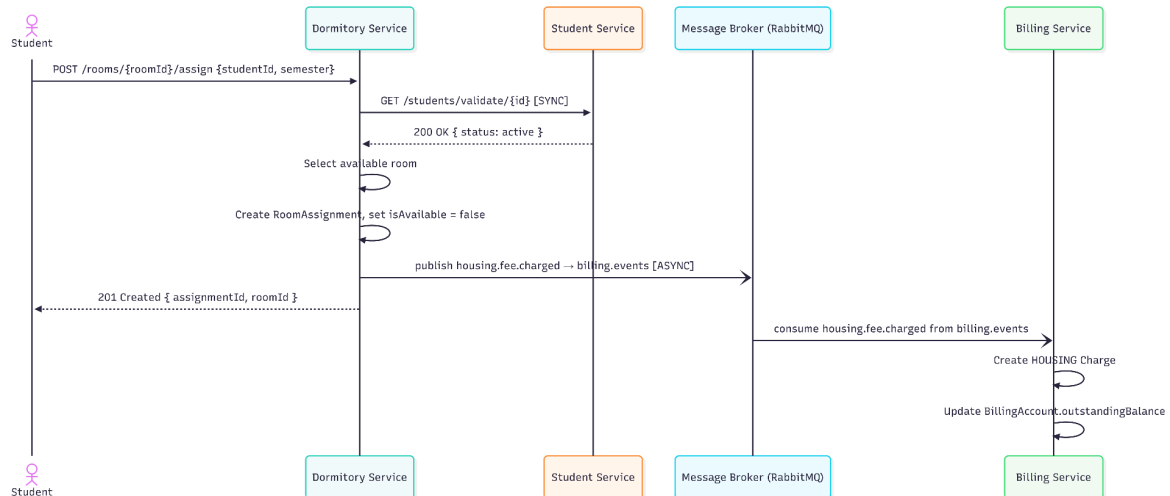
Step	From	To	Action / Event
1	Student (User)	Dormitory Service	POST /rooms/{roomId}/assign — student submits housing application with semester
2	Dormitory Service	Student Service	GET /students/validate/{id} — synchronous REST call to confirm student is active (R19)
3	Student Service	Dormitory Service	Returns 200 OK {status: active} or 404/403 error
4	Dormitory Service	dormitory_db	Selects available room, creates RoomAssignment record, sets DormitoryRoom.isAvailable = false (R20)
5	Dormitory Service	Message Broker	Publishes housing.fee.charged event to topic: billing.eventsPayload: { studentId, roomId, semester, amount }
6	Dormitory Service	Student (HTTP)	Returns 201 Created { assignmentId, roomId, semester }
7	Billing Service	Message Broker	Consumes housing.fee.charged event from billing.events [ASYNC] (R21)
8	Billing Service	billing_db	Creates HOUSING Charge record. Updates BillingAccount.outstandingBalance.

Data Tables Updated:

- dormitory_db — DormitoryRoom: isAvailable set to false
- dormitory_db — RoomAssignment: new record created (status: ACTIVE)
- billing_db — Charge: new HOUSING fee charge row inserted
- billing_db — BillingAccount: outstandingBalance updated

Event Details:

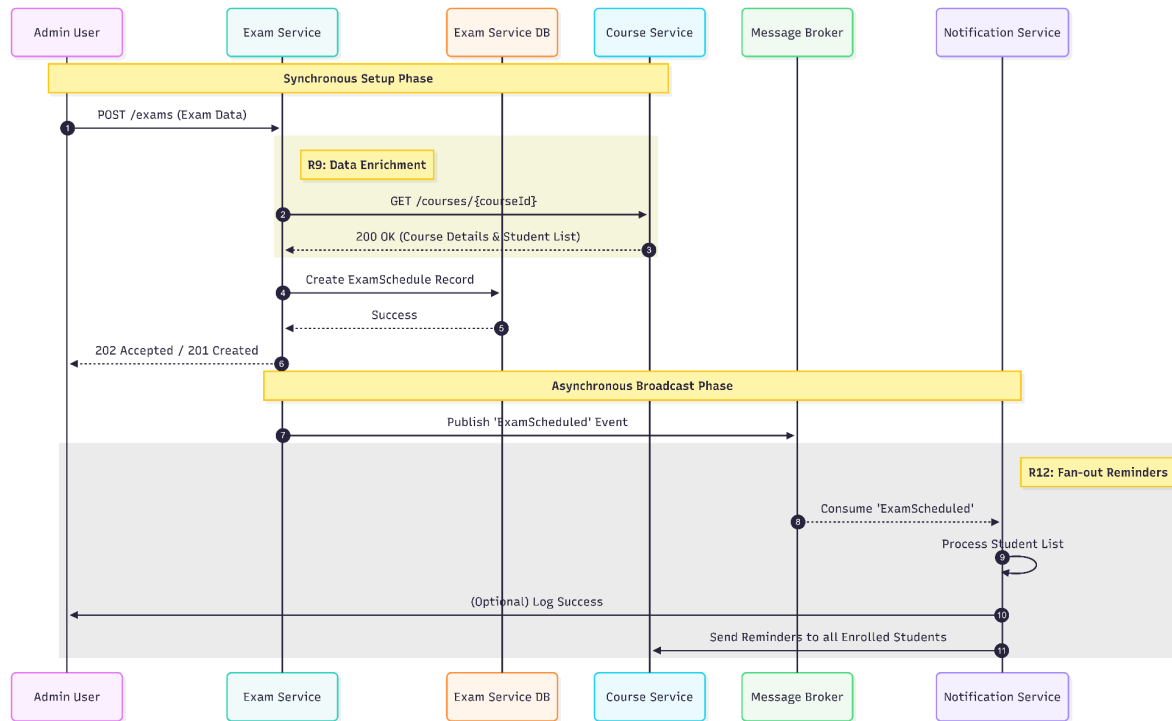
Event	Topic / Channel	Payload
housing.fee.charged	billing.events	{ studentId, roomId, semester, amount, chargeType: 'housing' }



Workflow 3: Exam Schedule Creation & Notification (Asynchronous interaction)

1. Admin sends POST /exams to **Exam Service**.
2. **Exam Service** makes a synchronous REST call to **Course Service** (GET /courses/{courseId}) to retrieve course details and enrolled students (R9).
3. **Exam Service** creates an ExamSchedule record.
4. **Exam Service** publishes the event ExamScheduled to the message topic.
5. **Notification Service** consumes the event and sends reminders to all enrolled students (R12).

Services involved: Exam Service (producer), Course Service (sync call), Notification Service (consumer). **Data tables updated:** ExamSchedule table in Exam Service DB.



Workflow 4: Tuition Billing Calculation

This workflow is triggered by the Billing Service (via admin request or automated scheduler) to calculate a student's tuition based on enrolled credits for the current semester. This is a fully synchronous workflow — no async events are used, as tuition calculation requires immediate consistency.

Services Involved:

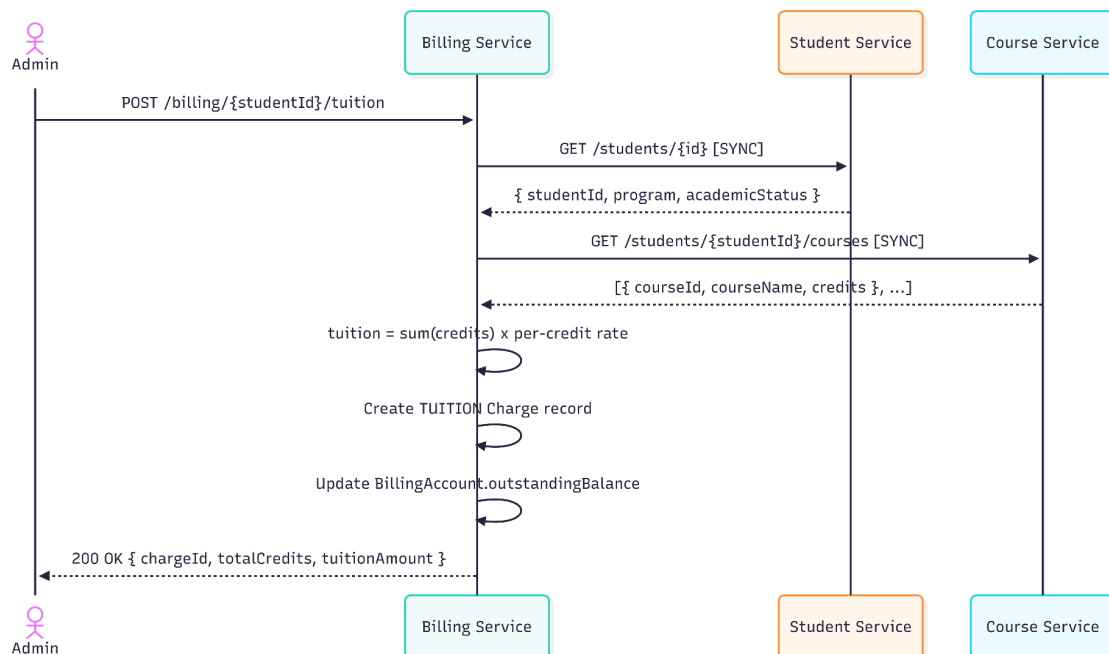
- Billing Service — orchestrator
- Student Service — student info retrieval (synchronous)
- Course Service — enrolled credits retrieval (synchronous)

Interaction Sequence:

Step	From	To	Action / Event
1	Admin / Scheduler	Billing Service	POST /billing/{studentId}/tuition — triggers tuition calculation for the semester (R27)
2	Billing Service	Student Service	GET /students/{id} — synchronous REST call to retrieve student academic info and confirm active status
3	Student Service	Billing Service	Returns 200 OK { studentId, program, academicStatus }
4	Billing Service	Course Service	GET /students/{studentId}/courses — synchronous call to retrieve all enrolled courses and credit counts (R27)
5	Course Service	Billing Service	Returns list of enrolled courses with credit values per course
6	Billing Service	billing_db	Calculates tuition = totalCredits × per-credit rate. Creates TUITION Charge record. (R27)
7	Billing Service	billing_db	Updates BillingAccount.outstandingBalance.
8	Billing Service	Admin / Scheduler	Returns 200 OK { chargeId, totalCredits, tuitionAmount }

Data Tables Updated:

- billing_db — Charge: new TUITION charge row with calculated amount
- billing_db — BillingAccount: outstandingBalance updated



5. Integration Mechanisms

5.1 Synchronous Communication

Services communicate synchronously using HTTP REST calls for operations that require an immediate response (e.g., identity verification, data retrieval for calculation).

Caller Service	Called Service	Purpose
Course Service	Student Service	Verify student identity and eligibility before course enrollment (R5)
Exam Service	Course Service	Retrieve course details and enrolled students for exam schedule creation (R9)
Library Service	Student Service	Authorize student identity before allowing book borrowing (R14)
Dormitory Service	Student Service	Verify student identity and active status before room assignment (R19)
Meal Plan Service	Student Service	Verify student identity before meal plan subscription (R23)
Billing Service	Student Service	Retrieve student info for billing record creation and tuition calculation (R26, R27)
Billing Service	Course Service	Get enrolled credit count per course for tuition calculation (R27)
Notification Service	Student Service	Retrieve student contact info (email/in-app) before message delivery (R31)

5.2 Asynchronous Communication

Events are published to a message broker (RabbitMQ). This decouples producers from consumers and improves system resilience — producing services are never blocked by downstream processing.

Event	Topic	Producer	Consumer(s)
enrollment.confirmed	course.events	Course Service	Notification Service (R8)
course.dropped	course.events	Course Service	Notification Service (R7)
exam.scheduled	exam.events	Exam Service	Notification Service (R12)
book.overdue	library.events	Library Service	Billing Service, Notification Service (R16, R17)
housing.fee.charged	billing.events	Dormitory Service	Billing Service (R21)
mealplan.fee.charged	billing.events	Meal Plan Service	Billing Service (R25)

6. Implementation Responsibilities

Sudias

I am responsible for implementing the following two microservices:

- **Course Service** (full REST API + own persistent database + participation in both implemented workflows)
- **Exam Service** (full REST API + own persistent database + participation in both implemented workflows)

Both services expose a non-trivial REST API (6 endpoints each), manage their own persistent data, participate in at least two interactions with other services, and are part of the implemented system workflows.

Services that will be implemented by the team (summary for the whole document):

- All business services listed in Assignment 1 (Student, Course, Exam, Library, Dormitory, Meal Plan, Billing, Notification).
- No services are replaced by shared components in this team.

Workflows that will be implemented:

- Course Enrollment workflow (synchronous + asynchronous)
- Exam Schedule Creation & Notification workflow (synchronous + asynchronous)

Hashim

I am responsible for implementing the following two microservices:

- Student Service: full REST API (6 endpoints) + own persistent database. Acts as the central identity provider; participates in at least five interactions with other services (Course, Library, Dormitory, Meal Plan, Billing, and Notification all call it). Participates in both implemented workflows: Workflow 1 (direct actor) and Workflow 2 (called synchronously by Notification Service for contact info retrieval).
- Notification Service: full REST API (7 endpoints) + own persistent database. Consumes asynchronous events from Course Service (course.events) and Exam Service (exam.events) and Library Service (library.events). Calls Student Service synchronously to retrieve contact info before delivery. Participates in Workflow 2 as the primary actor.

Both services expose a non-trivial REST API, manage their own persistent data, participate in at least two interactions with other services, and are part of the implemented system workflows.

Calvin

I am responsible for implementing Library Service & Billing Service:

Library Service: Exposes 8 REST endpoints for book catalog management and borrowing lifecycle. Manages Book and BookLoan entities. Calls Student Service synchronously for authorization. Publishes book.overdue event to library.events topic; consumed by both Billing Service (fine charge) and Notification Service (overdue alert). Primary orchestrator in Workflow 3.

Billing Service: Exposes 6 REST endpoints for charge management, payment simulation, and account overview. Manages BillingAccount, Charge, and Payment entities. Calls Student Service and Course Service synchronously for tuition calculation. Consumes book.overdue events from library.events, and housing.fee.charged and mealplan.fee.charged events from billing.events topic. Orchestrator in Workflow 4; event consumer in Workflows 2 and 3.

Daboikiabo

I am responsible for implementing Meal Service and Dormitory Service

Dormitory Service: Full REST API + own persistent database (dormitory_db). Calls Student Service synchronously for identity verification. Publishes room.assigned and room.assignment.cancelled events to dormitory.events topic; consumed by both Billing and Notification services.

Meal Plan Service: Full REST API + own persistent database (mealplan_db). Calls Student Service synchronously for eligibility checks. Publishes mealplan.subscribed and mealplan.cancelled events to mealplan.events topic; consumed by both Billing and Notification services.

6.3 Workflows to Be Implemented

The following four workflows will be implemented during the project phase:

Workflow	Integration Style	Services Involved
Workflow 1: Student Course Enrollment	Sync + Async	Course Service, Student Service, Notification Service
Workflow 2: Dormitory Room Assignment & Billing	Sync + Async	Dormitory Service, Student Service, Billing Service
Workflow 3: Exam Schedule Creation & Notification	Sync + Async	Exam Service, Course Service, Notification Service
Workflow 4: Tuition Billing Calculation	Sync only	Billing Service, Student Service, Course Service